



LightML: A Photonic Accelerator for Efficient General Purpose Machine Learning

Liang Liu
University of Pittsburgh
Pittsburgh, Pennsylvania, USA
lil125@pitt.edu

Sadra Rahimi Kari
University of Pittsburgh
Pittsburgh, Pennsylvania, USA
sadra.rahimi@pitt.edu

Xin Xin
U. of Central Florida
Orlando, Florida, USA
xin.xin@ucf.edu

Nathan Youngblood
University of Pittsburgh
Pittsburgh, Pennsylvania, USA
nathan.youngblood@pitt.edu

Youtao Zhang
University of Pittsburgh
Pittsburgh, Pennsylvania, USA
zhangyt@cs.pitt.edu

Jun Yang
U. of Pittsburgh
Pittsburgh, Pennsylvania, USA
juy9@pitt.edu

Abstract

The rapid integration of AI technologies into everyday life across sectors such as healthcare, autonomous driving, and smart home applications requires extensive computational resources, placing strain on server infrastructure and incurring significant costs.

We present LightML, the first system-level photonic crossbar design, optimized for high-performance machine learning applications. This work provides the first complete memory and buffer architecture carefully designed to support the high-speed photonic crossbar, achieving over 80% utilization. LightML also introduces solutions for key ML functions, including large-scale matrix multiplication (MMM), element-wise operations, non-linear functions, and convolutional layers. Delivering 325 TOP/s at only 3 watts, LightML offers significant improvements in speed and power efficiency, making it ideal for both edge devices and dense data center workloads.

CCS Concepts

- **Hardware** → **Emerging optical and photonic technologies**;
- **Computing methodologies** → *Neural networks*.

Keywords

Optical Computing Crossbar, Coherent Photonic Multiplication, Ultralow-Energy Machine Learning

ACM Reference Format:

Liang Liu, Sadra Rahimi Kari, Xin Xin, Nathan Youngblood, Youtao Zhang, and Jun Yang. 2025. LightML: A Photonic Accelerator for Efficient General Purpose Machine Learning. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3695053.3731053>

1 Introduction

The booming AI market is producing numerous technologies that are quickly adopted and deployed in people’s daily lives, spanning

application domains ranging from healthcare [2, 60, 66] and autonomous driving [44, 48] to smart homes [11, 50]. The sizes of these CNN, DNN, and Transformer-based AI models are rapidly expanding, demanding immense computational resources and pushing the server infrastructure to its limits, resulting in significant financial costs [5, 20, 76, 88]. Training and inference tasks for large language models can easily demand teraflops of computational power per inference task or training epoch [31]. Consequently, this “computational obesity” often restricts their deployment to large servers and data centers and frequently impedes user-side training and inference.

Computing with light offers a compelling alternative to conventional computing because optical interconnects, such as fibers and waveguides, are not constrained by energy efficiency issues related to Joule heating, RF crosstalk, and capacitance [33, 52, 65]. This eliminates the energy-bandwidth tradeoff that restricts the data transfer rate of electrical interconnects and allows an exceptionally high bandwidth density [8]. Various architectures have demonstrated the feasibility of photonic computing [3, 13, 18, 46, 68, 92].

While these properties benefit the efficiency of today’s machine learning applications, previous work has often focused on isolated physical aspects, lacking complete architectural support that integrates all necessary components. In particular, a few significant challenges remain to be addressed: 1. Existing work lacks a thorough memory solution to fully saturate the high data demand. For applications with complex data structures, such as large-scale matrix convolution, naive memory fetching schemes limit the performance of the crossbar arrays. 2. Current optical crossbar designs support limited operations, whereas general-purpose ML applications require a broader range of functionalities, including non-linear functions, element-wise operations, and normalization functions. 3. Prior designs did not fully exploit circuit-level optimizations, resulting in inefficiencies in basic operations.

In this paper, we introduce LightML, an electronic-photonic co-designed architecture that optimizes memory access and supports full-spectrum ML-specific operations to address existing gaps. The contributions of this work include:

- Pioneering the first system-level photonic crossbar architecture with a novel memory and buffer design that sustains high-speed data throughput.
- Integrating a non-linear function unit by utilizing phase modulators and achieving non-linear functions via the Fourier Series.



This work is licensed under a Creative Commons Attribution 4.0 International License. *ISCA '25, Tokyo, Japan*

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1261-6/25/06
<https://doi.org/10.1145/3695053.3731053>

- Efficiently implementing operations such as matrix transpose, batch normalization, and ReLU activation functions through simple and easy-to-deploy circuit-level designs.

The advantages of LightML include its computing speed and high energy efficiency, offering 325 TOP/s computing power with only 3 watts of power consumption. When including HBM, the system achieves a power efficiency of 17.1 TOP/s/W—13.6x higher than a GPU, and when excluding HBM (under equivalent configurations), it is up to 1.91x more efficient than state-of-the-art NVM-based crossbars. Additionally, with the proposed optimization, LightML can rival the NVIDIA A100 for ML model inference, providing up to 4x shorter latency. Furthermore, LightML delivers 5-bit precision while maintaining less than 3% inference accuracy loss compared to models in Floating Point. Therefore, LightML is an excellent choice for both edge devices, where energy efficiency is crucial, and data centers, where it can handle dense computing tasks efficiently.

2 Background

2.1 Photonic MAC Operation

Signed multiplication and addition can be performed optically using a simple beam splitter and differential photodetection. For multiplication, we make use of homodyne detection, which interferes with two optical fields with each other to achieve photo-electric multiplication at the quantum limit [27, 71]. When two coherent optical signals are incident on a 3dB coupler, which outputs two light beams with 50% power, in Figure 1a), the instantaneous differential output intensity will be:

$$I_+ - I_- = 2|xy| \sin(\Delta\phi)$$

where x and y are the electric field amplitudes and $\Delta\phi = \phi_x - \phi_y$ is the relative phase difference between the two optical signals. Differential detection removes the common mode intensities of the input signals ($|x|^2$ and $|y|^2$), leaving only the product of the two field amplitudes, $|xy|$. Because the interference also depends on the relative phase difference between the two input signals, it is possible to encode both positive and negative numbers (e.g., $-y = |y|e^{j\pi}$) and even complex numbers as recently demonstrated [38]. Since we encode data sequentially in the time domain, addition can be performed by temporal integration via charge accumulation on a capacitor, as shown in Figure 1b). Thus, the dot-product between vectors x and y can be accomplished optically on-chip with only a 3dB directional coupler and a few simple electrical components. This reduces both the input optical power and the ADC frequency by $1/N$, where N is the number of time-multiplexed elements (i.e., length of the input vectors), thus significantly improving the scalability and compute efficiency over weight-stationary optical approaches [89].

2.2 Optical Modulator

The Michelson interferometric modulator (MIM) design [62] is comprised of a segmented intensity modulator with binary code weighting [83] (Figure 1d). By applying voltage to the segmented yellow cells, we can precisely control both the phase and amplitude of the modulated light. A π phase shift ($y e^{j\pi} = -y$) is used to encode the sign of the input signal, while the amplitude is modulated linearly in proportion to the cell length. Different cell lengths are employed to represent the MSB and LSB. The use of segmented modulators

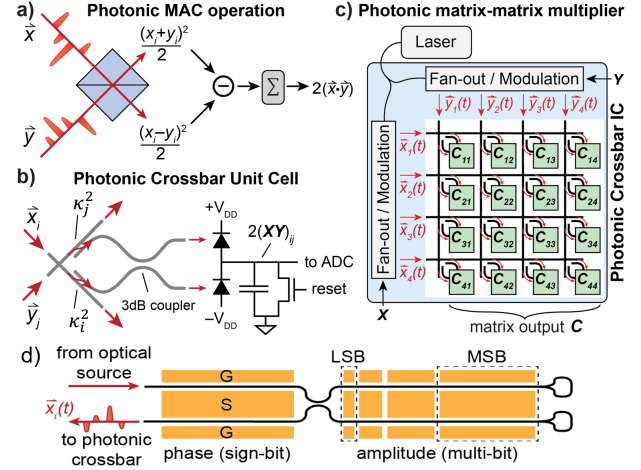


Figure 1: a) Photonic MAC uses a beam splitter and differential detection. b) Circuit-level implementation of a) using an on-chip beam splitter and integrating circuit. c) Photonic crossbar architecture for matrix-matrix multiplication comprised of an array of dot-product unit cells (green). d) Illustration of optical DAC comprised of a segmented Michelson modulator and phase shifter.

allows for direct electro-optic (E-O) conversion of an N -bit binary value to an analog amplitude without the need for a dedicated DAC for each modulator. This can achieve high E-O conversion efficiency (< 250 fJ/b) and better linearity than conventional optical modulators + electronic DAC designs [51, 83]. Additionally, the folded cavity design reduces footprint and capacitance by $\sim 2x$ [62].

2.3 Photonic Crossbar

To scale this concept beyond single dot-product operations, we can create an array of dot-product unit cells, each coupled to row and column waveguides via directional couplers with a predetermined splitting ratio (κ_i^2 and κ_j^2) as illustrated in Figure 1c). This allows optical fan-out in a compact, two-dimensional array and amortizes the energy required for modulation. Similar arrays have been experimentally demonstrated with a large number of unit cells (e.g., 64×64 in [74] and 128×128 in [93]) in other contexts such as LI-DAR [64, 93], photonic phased arrays [74], and in-memory photonic processors [18]. The crossbar can perform integer matrix-matrix multiplication (MMM), $C = XY$, where $X \in \mathbb{N}^{D \times P}$ and $Y \in \mathbb{N}^{P \times D}$, P are the number of pulses temporally streamed into the array from memory, and D can be the dimension of the crossbar array (up to 128). Previous work [89] shows that when the number of pulses, P , achieves 1024, the computational efficiency (TOPs/s/W) reaches the maximum. Our photonic crossbar architecture allows larger-scale MMMs with tiling, as we explore in Section 6.

The remarkable speed primarily stems from the ability to perform a dot product *at each crosspoint* of the photonic crossbar at speeds exceeding billions of multiply-accumulate (MAC) operations per second per crosspoint. In contrast, in conventional resistive memory-based crossbars, a dot product is executed in each *column*

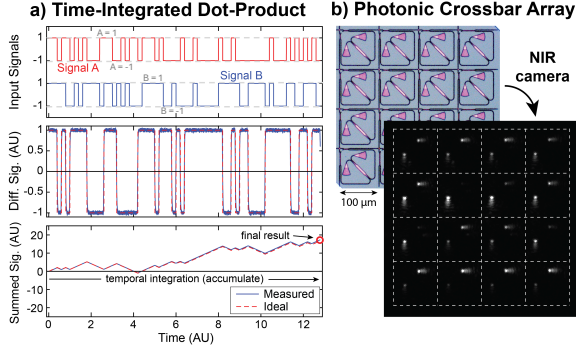


Figure 2: a) Measured dot-product between two bipolar vectors (top). Difference signal measured by balanced photodetection (middle). Integrated difference signal gives the final result with a 3.6% error (bottom). b) Microscope image of 4 × 4 crossbar array (top) and captured NIR image of crossbar output (bottom). Reproduced from [38].

or row at speeds ranging from tens to a few hundred megahertz, limited by interconnect capacitance and analog noise.

3 Preliminary Result and Fabrication Feasibility

3.1 Preliminary Result

We have demonstrated the feasibility of our proposed architecture at both the unit cell and crossbar array levels [38]. A 4×4 homodyne multiplier prototype was fabricated in the laboratory. The prototype utilizes non-resonant thermo-optic Mach-Zehnder interferometer (MZI) modulators with a tunable fiber laser source emitting 1550 nm light. Each cell in the array consists of two directional couplers to evenly distribute power from the row and column waveguides. A compact 3dB directional coupler connects to two output ports, which are further coupled to a Near-Infrared (NIR) image sensor via grating couplers. Results from a single unit cell are presented in Figure 2a). In this demonstration, two modulators are used to encode the phase of each signal, enabling the computation of the dot product between two 64-element bipolar vectors. The differential signal from the two output grating couplers performs multiplication (middle panel), while temporal integration achieves accumulation (bottom panel). Additionally, we successfully demonstrated optical fan-out and NIR imaging of a passive 4x4 photonic crossbar array in Figure 2b). Post-processing the captured NIR image allows the extraction of MMM results, showcasing the scalability and practicality of our approach.

3.2 Error Modeling

The precision of the photonic crossbar is sensitive to noise and signal degradation. To assess the feasibility of industrial fabrication, we model the impact of noise from various fabrication non-idealities based on our photonic crossbar design. In Figure 3a), we present a preliminary error propagation analysis of optical MAC operations in a single unit cell. Assuming two digital input signals of dimension N , $\mathbf{x}, \mathbf{y} \in \mathbb{N}^N$, the ideal MAC output is $\mathbf{x}^T \mathbf{y}$. To evaluate the relative error, ϵ , between the ideal output and the measured result from the

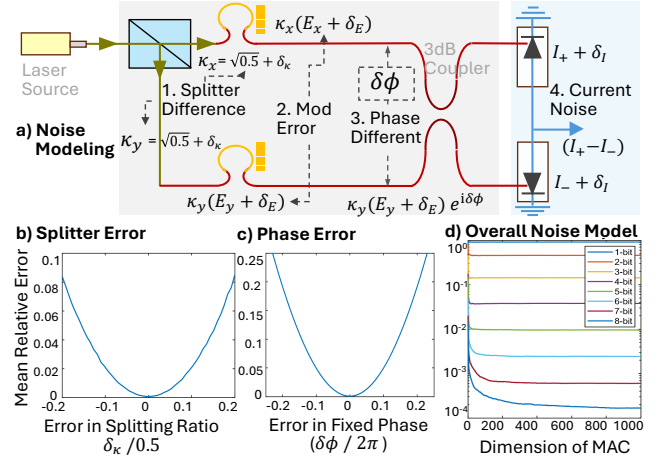


Figure 3: a) Model of noise and fabrication error within each unit cell. b) and c) Ablation study of splitting ratio and phase noise. d) Overall Noise Modeling.

ADC, we use the following equation:

$$\epsilon = \left(\mathbf{x}^T \mathbf{y} - (\mathbf{x}^T \mathbf{y})_{\text{measured}} \right) / \mathbf{x}^T \mathbf{y}$$

The measured output, $(\mathbf{x}^T \mathbf{y})_{\text{measured}}$, is the voltage measured from the accumulated circuit, which is $c_v \cdot \int \frac{1}{2} (I_+(t) - I_-(t)) dt$. Here, the integral term $\int (I_+ - I_-) dt$ represents the accumulated voltage over the MAC period, and c_v is a calibration coefficient that scales the accumulated current to the ADC output. The noise modeling results shown in Figure 3 have been tailored for our architecture, drawing upon seminal theories of homodyne detection [58, 90] as well as recent developments in coherent photoelectric multiplication for AI processing [26].

1. Beam Splitters During optical fan-out, 50:50 beam splitters divide the laser into $2N$ beams with equal intensity before fanning-out these beams to each unit cell in a given row or column via directional couplers [89]. However, imperfections in splitting ratios during fan-out and distribution of the signals introduce power imbalances as the crossbar size increases. Each beam passes through $\log_2 2N$ 50:50 beam splitters and up to N directional couplers, accumulating a fixed power imbalance due to deviation from the ideal splitting ratio. We model the power splitting ratio of each 50:50 splitter as $\kappa_x, \kappa_y = \sqrt{0.5} \pm \delta_k$, while the coupling coefficients for each directional coupler in the crossbar can be written as $\kappa_n^2 = \kappa_{n+1}^2 / (1 + \kappa_{n+1}^2) + \delta_k^2$ where δ_k is the fixed fabrication error for each splitter/coupler sampled from a Gaussian distribution.

2. Modulation Error: Optical amplitude noise arising from analog electrical and optical noise in the modulators, driving circuitry, and the laser itself is present when encoding the optical inputs. The light intensity after modulation is modeled as $E_x + \delta_E$ and $E_y + \delta_E$, where $E_x \propto x$ and $E_y \propto y$ are the ideal electric field intensities, and δ_E is Gaussian noise.

3. Phase Shift: Variations in the optical path lengths between the two signals, x and y , introduce fixed phase alignment errors. These errors occur due to minor differences in the physical paths traversed by the signals and can vary with temperature. Typically, the optical signals travel distances ranging from 1 to 10 mm between the modulator and the 3dB coupler. We model this phase noise as

$e^{i\delta_\phi}$, where δ_ϕ follows a Gaussian distribution, representing the random variations in phase alignment.

4. Photon Detector: Noise also occurs during optical-to-electrical conversion. The generated current is $I = \eta \cdot \frac{e}{h\nu} |E|^2$, where $\frac{e}{h\nu} |E|^2$ is the photon energy, and η is the photon detector's conversion efficiency (typically between 0.5 and 0.8). We model photon detector noise as Gaussian noise δ_I , affecting the output as $I_+ + \delta_I$ and $I_- + \delta_I$.

5. Thermal Condition: Photonic devices are sensitive to thermal variations because the refractive index of silicon changes slightly with temperature. In an RC circuit, thermal (Johnson–Nyquist) noise [41] induces voltage fluctuations across the capacitor, which can be expressed as: $V_{\text{rms}} = \sqrt{k_B T / C}$, where k_B is the Boltzmann constant, T is the absolute temperature, and C is the capacitance. For instance, with $C = 15\text{fF}$ (the same as DRAM) at room temperature (300 K), the voltage fluctuation is about 0.5 mV, which has only a minimal effect on our error model. Furthermore, our platform minimizes thermal variation by consuming less than 20W (including HBM2E), resulting in negligible temperature increases.

Other sources of noise, such as those from ADC readers, are negligible due to the maturity of these technologies. In Figures 3b) and 3c), we plot the relative dot-product error due to errors in the splitting ratios and phase error, respectively, while holding other noise sources constant. A smaller relative error ensures higher bit precision. For instance, a relative error below $1/2^4 = 0.0625$ allows for a 5-bit precise MAC operation (4 bits for magnitude and 1 bit for the sign). It is important to note that errors in both the splitting ratio and phase can be removed if: 1) they are constant after fabrication and 2) can be determined through an initial calibration step. In Figure 3d), we apply the Monte Carlo method to model the entire noise system. All noise sources follow Gaussian distributions with mean $\mu = 0$ and standard deviation $\sigma = 1/2^{N_b}$, where $N_b = \{1, 2, \dots, 8\}$ represents different precision levels for devices (e.g., modulators, photon detectors). The relative error is plotted against the MAC operation dimensions, showing that the error decreases as the temporal dimension increases. Our previous work [67] analyzed the accuracy degradation of various ML models under different error configurations. In this work, we also simulated the error model by injecting Gaussian noise during inference. As shown in Table 6, the resulting accuracy drop remains within approximately 1% to 2%.

3.3 Industrial improvement

Our laboratory prototype is limited by the available fabrication technologies. However, industries with state-of-the-art fabrication techniques [36, 81, 84] can achieve higher precision, operating frequencies, and address many limitations.

Optical Modulator: Existing work [84] has demonstrated a 20 GHz optical modulator fabricated using 40 nm technology, achieving both sign encoding and 32-level magnitude resolution. This corresponds to a 6-bit precision modulator.

Phase Alignment: For optical devices operating at a 1550 nm wavelength, the relative optical path length must be controlled to within $1550/(n_{\text{eff}} \cdot 2^4) \approx 50\text{ nm}$ to maintain 4-bit amplitude and 1-bit sign. Achieving this requires laser trimming individual unit cells in the photonic crossbar array, as discussed extensively in [89]. Alternatively, localized thermal tuning has been demonstrated for

post-fabrication trimming of silicon photonic devices [36]. Our lab is currently exploring using low-loss phase-change materials for laser trimming, building on methods shown in [81].

Splitting Ratio: To address optical losses and imperfections in splitting or coupling ratios and achieve 5-bit precision, the splitting ratio should maintain $\delta_k^2 < 0.5/2^5$. A one-time calibration of the crossbar array can be performed. This calibration applies a scaling factor to each unit cell, compensating for fabrication variations and enabling the scalability of larger crossbar arrays.

Using SOTA technology, the precision of non-ideal components can exceed 5 bits, ensuring $\sigma < 1/2^5$. In Figure 3d), the relative error under 5-bit precision approaches $10^{-2} \approx 2^{-6}$, supporting results with approximately 6 bits of magnitude and 1 bit for the sign. Our design employs a conservative strategy, targeting 4 bits for magnitude and 1 bit for the sign, with potential scalability to 8 bits in future iterations. For instance, [92] experimentally demonstrated 8-bit precision across 1,000 optical MAC operations using high-performance foundry-fabricated modulators. Additionally, [13] adopts the Residue Number System (RNS) to address precision limitations in photonic computing, enabling the system to operate reliably at lower precision.

4 Advantages of Photonic Computing Core

As noted earlier, the feasibility of fabricating optical crossbars has been demonstrated by our lab prototypes and refined by the broader research community. Building on this foundation, we will develop a fully functional accelerator for machine learning, called “LightML”. Key advantages over prior accelerators include:

1. Significant Speed Boost over Resistive Crossbars. LightML offers significant advantages over resistive memory crossbars for matrix multiplication, including orders-of-magnitude improvements in speed and energy efficiency. This speed stems from the photonic crossbar's ability to perform N^2 dot-products simultaneously at each crosspoint, enabling true MMM, unlike resistive memory crossbars that handle only N dot-products at a time for matrix-vector multiplication (MVM). LightML computes using electro-optical modulators that operate at the GHz range, while resistive memories require multiple ADC-intensive iterations for full matrix operations. Further, LightML does not require high-current and slow reprogramming of weights for ML models, which is much more efficient and low energy than resistive memory crossbars.

2. Support for Various Non-Linear Functions. There have been prior studies on implementing complete ML models, such as MNIST [81], using optical crossbars specifically for MMM or MVM [12, 24, 86]. In these works, operations beyond MMM are still handled by a host processor. However, optical signals inherently encode both amplitude and phase information, and this phase information enables straightforward computation of functions like sin and cos. These functions can be used to derive non-linear functions such as Exponential, Sigmoid, and tanh, which are critical for ML applications (details are provided in Section 6.2). Existing accelerators require additional functional units to compute these non-linear operations, leading to significant data transfer overheads. In this paper, we demonstrate how both MMM and non-MMM operations can be executed entirely within the optical crossbar. To the best of our

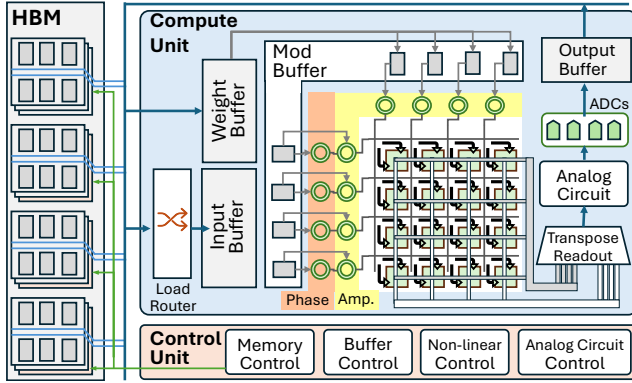


Figure 4: Overview of LightML.

knowledge, this is the first work to establish a photonic computing unit capable of general-purpose ML acceleration.

3. High Energy Efficiency. Although the modulators can operate at tens of GHz, our prior studies have shown that the optical crossbar, which includes the laser source, modulators, and readout circuits, consumes less than 1W [89]. As demonstrated in the evaluations section, these combined advantages enable LightML to rival the NVIDIA A100 GPU in computing power while consuming $\frac{1}{13}$ of the energy, and achieve 1.91x power efficiency compared to the SOTA photonic and PCM-based crossbars [19, 47, 53].

5 Overview

5.1 LightML Architecture

In Figure 4, we show the architectural overview of LightML. The proposed architecture comprises the core compute unit, an optical crossbar, supporting analog circuitry, memories and buffers, and control units, as illustrated in Figure 4. The compute unit performs MMM, MVM, nonlinear functions, and element-wise operations. The memories and buffers are designed to sustain the high input/output data rates required by the compute unit, while the control unit orchestrates complex functions composed of multiple primitive operations, as listed in Table 1.

Computing Unit: The crossbar is equipped with 2×128 modulators for amplitude modulation (in both dimensions) and 1×128 phase modulation (in the horizontal dimension), with the latter primarily used for computing non-linear functions. Each crosspoint includes an accumulation capacitor connected to analog-to-digital converters (ADCs) for processing readouts, as shown in Figure 1b). Column and row selectors are employed to reduce the number of ADCs by time-multiplexing them, allowing us to explore trade-offs between area, power, and latency. The ADCs and supporting circuitry will be further detailed to handle frequent operations such as divisions and transpositions (See Section 6.4).

Table 1: AI Functions Supported by LightML.

Class	Function Name
Element-Wise Ops	Add, Sub, Mul, Scaling
Matrix/Vector Ops	Sum, Mean, Dot, MVM, MMM
Nonlinear Ops	Sin, Log, Exp, Tanh, Sigmoid
Normalization Ops	BatchNorm, LayerNorm
Systolic Ops	Linear, Conv2d

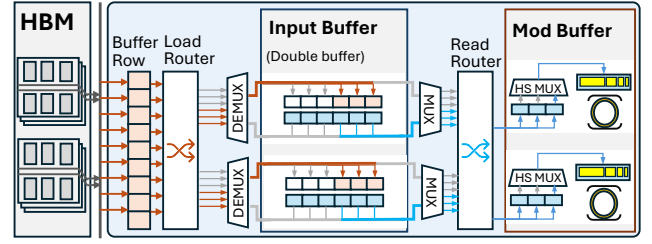


Figure 5: Design Scheme of On-chip Input Buffers.

Control Unit: The control unit consists of several components: a memory controller for address translation, a buffer controller to manage load/store routers and pipelining buffers, a nonlinear controller equipped with internal registers for storing coefficients and parameters required for non-linear functions, and an analog controller responsible for overseeing the circuits around the ADCs to perform regular, transpose, and scaling operations.

Memory and On-chip Buffer: The memories and buffers are designed to meet the high bandwidth requirements of the compute unit, for a scalable crossbar, such as a 128×128 array with 2×128 modulators operating at 12 GHz, requiring $2 \cdot 128 \cdot 12G = 3TB$ data per second to fully utilize the crossbar at the peak performance.

In Figure 5, we illustrate the data flow from memory to the modulator. LightML adopts a 2-stack High-Bandwidth Memory (HBM2E) configuration, providing up to 920 GB/s of memory bandwidth. For each memory cycle, HBM2E fetches 1KB of data for the chip. We implement a 1KB buffer row to load this data on the chip.

We implement three on-chip buffers: input, weight, and output. To support continuous data flow to the optical modulators, the input buffer uses a double-buffer scheme—one buffer loads from HBM while the other drives the modulators, alternating each cycle. This ensures data delivery within a single dot-product duration. As shown in Figure 5, top blocks receive data, and bottom blocks feed the modulators. DEMUXs and MUXs control write and read paths, respectively. The input buffer consists of 128 lines for 128 modulators, each with 1KB of SRAM, totaling 128 KB.

Data from the loaded row (1KB) is distributively written into all 128 lines of the input buffer, with each line sequentially receiving 8B data. We implement a load router to control the data writing addresses, ensuring consistent addressing logic across all 128 lines. The weight buffer operates similarly to the input buffer but uses a single buffer architecture. When fetching the weights, the modulator operation is halted until the process is complete.

5.2 Flexibility in General AI Functions Support

AI models typically consist of complex modules—including Multi-Layer Perceptrons, convolutional layers, attention mechanisms, and various nonlinear activations. However, many of these modules can be decomposed into a small set of fundamental operations, such as MMM, MVM, element-wise operations, and matrix transpositions. For instance, the Transformer’s attention mechanism involves MMM, softmax (a nonlinear function), element-wise scaling, addition, and matrix transposition [79]. We conduct a comprehensive analysis of these complex operations and identify common primitive components. The supported functions are summarized in Table 1. A majority of those primitive components can be efficiently implemented on our LightML, while the remaining operations are

primarily memory-intensive. Implementing these operations enables acceleration for a wide range of AI models.

6 Common Modules for ML Applications

6.1 Quantization

To address the precision constraint, this work leverages the standard integer quantization machinist [21, 82]. The integer quantization utilizes batch-wise scaling factor, s , to map the real value $x \in [-S, S]$ into its 5-bit integer format, $\hat{x} \in [-16, 16]$:

$$\text{quant: } \hat{x} = [x] / s, \quad \text{dequant: } x = s \cdot \hat{x}$$

The scaling factor s is computed by $s = S/16$, where S is the real-valued range of the input, and 16 is the integer range limit. During the MAC operation, all elements within each input matrix/vector $\hat{\mathbf{x}}$ and $\hat{\mathbf{w}}$ share the same scaling factors, s_x and s_w . The output $\hat{y}$ is

$$\hat{y} = \sum_i^N \hat{w}_i \hat{x}_i = \sum_i^N s_w w_i \cdot s_x x_i = s_w s_x \cdot y$$

Hence, the integer output $\hat{y}$ has the combined scaling factor $s_w s_x$. In the worst-case scenario, $\hat{y}$ may require $2 \times 5 + \log_2(N) - 1$ bits to maintain accuracy. In conventional digital computing, quantized outputs are typically stored in higher-precision formats, a new scaling factor is derived, and the data is then downscaled to 5 bits, which is deployed in the cuBLAS library [56].

In LightML, downscaling is efficiently handled by our ADC scaling unit (see Section 6.4). First, we perform a one-time calibration of the laser power so that the maximum MAC output $S_{\max}$ —obtained when 1,024 inputs are each set to the maximum value (+16)—corresponds to 1V in the ADC readout. For a sequence of outputs with an individual scale, S' , we will adjust the ADC's reference voltage to $S' / S_{\max}$ V, fitting the outputs within the appropriate range.

6.2 Non-Linear Function

Nonlinear functions, typically serving as activation functions in neural networks, vary by model. For instance, Long-Short-Term Memory (LSTM) [30] and Recurrent Neural Networks (RNN) [63] use $\tanh$, while transformers [79] use the *Sigmoid* function (via SoftMax). Traditionally, they are implemented using math libraries with parallelism support, when available, on CPUs and GPUs. In ML accelerators, these functions are typically executed on dedicated hardware, which lacks the flexibility to easily adapt to new functions and algorithms. There is also prior work [73] that uses a diode array to simulate nonlinear functions; however, it supports only a single function and requires a carefully tailored sequence, leading to increased latency.

In our design, we implement nonlinear functions via the Fourier Series, and we use the phase modulator to generate sin or cos function outputs. The phase modulator calibrates the waveguide to ensure the phase difference between two light beams is 0 or π . To compute the nonlinear function, we couple two light beams: one containing only phase information and the other containing only amplitude information. The phase input, ϕ , is represented as 5-bit data, dividing the phase into 32 intervals between $-\pi$ and π . The amplitude input is encoded as 5-bit data, a . After coupling, the current readout will be proportional to $a \sin(\phi)$. For any nonlinear

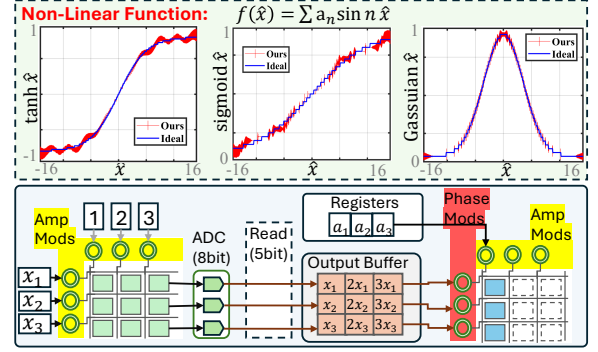


Figure 6: Implementation for Nonlinear Functions.

function, $f(x)$, we can decompose it as a Fourier Series:

$$f(x) = \sum_{k=1}^N a_k \cdot \sin(2\pi kx/L) + b_k \cdot \cos(2\pi kx/L)$$

where the input, x , is within the interval $[-L, L]$, and N is the degree of the series (number of terms). a_k and b_k , are invariant to the input, x , and can be computed by:

$$a_k = \int_{-L}^L f(t) \sin\left(2\pi \frac{kt}{L}\right) dt, \quad b_k = \int_{-L}^L f(t) \cos\left(2\pi \frac{kt}{L}\right) dt$$

These coefficients are unique for each nonlinear function. For an odd function $f(x)$, the symmetric integral yields $b_k = 0$, and for an even function, $a_k = 0$. We preload the coefficients into the controller's register file. For a 5-bit precision digital input, the maximum degree for the Fourier Series is 32, and the register stores up to 64 5-bit coefficients for each nonlinear function.

Since the Fourier Series is periodic, it regulates the input within a specific interval. For each nonlinear function, we define a corresponding effective interval. The computation allows the input to be within this interval, $-L \leq x \leq L$ (e.g., using $L = 4$ for the Sigmoid function), and all $x > L$ will be capped to L . The quantized input is a 5-bit data using $\hat{x} = \lfloor 16 \cdot \frac{x}{L} \rfloor$. For all multipliers of $\hat{x}$, e.g., $2\hat{x}, 3\hat{x}, \dots$, larger than 16, we modulate them by 32, which can be achieved by reading the last 5 bits of the 8-bit readouts.

In Figure 6, we illustrate the flow of nonlinear functions in three steps: 1. compute the multiplier: The input x is modulated through an amplitude modulator and multiplied by the factors $1, 2, 3, \dots, N$; 2. intermediate result measurement: The output is measured using 8-bit ADCs, with the last 5 bits extracted as the intermediate result and stored in the output buffer; 3. phase encoding and final multiplication: All multipliers $\hat{x}, 2\hat{x}, \dots, N\hat{x}$ are encoded as phase information and multiplied by the corresponding coefficients $a_1, a_2, \dots, a_N$. LightML is capable of batch computing up to 128 inputs simultaneously, significantly enhancing throughput for nonlinear operations. At the top of Figure 6, we use the 5-bit precision error model from Section 3.2 to simulate three non-linear function: Sigmoid, tanh, and e^{-x^2} (Gaussian). The blue curve represents the ideal 5-bit output, while the red error bars indicate the 3-sigma error range of our implementation. The average error rate is 4.2%, which is generally tolerable in most machine learning applications.

6.3 Element-Wise Operation

Element-wise operations, such as addition and multiplication, apply arithmetic operations to corresponding elements of two or more vectors or matrices. They are not only common but also essential in

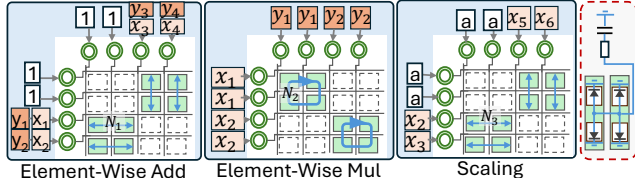


Figure 7: Implementation for Element-Wise Operations.

many AI models, particularly in skip connections, backpropagation, normalization, and attention mechanisms. Note that element-wise arithmetic is a special case of the dot product. These operations can be implemented straightforwardly, but efficiency and accuracy must be carefully considered, as illustrated below.

Addition: On the left of Figure 7, we illustrate the addition of two vectors, x and y . The inputs x_i and y_i are encoded as two consecutive pulses from a vertical or horizontal modulator, respectively. A modulator in the perpendicular direction generates constant light with unit amplitude, resulting in the output $1 \cdot x + 1 \cdot y$. However, computing $x_i + y_i$ at a single crosspoint does not generate sufficient charge accumulation in the capacitor. To rapidly accumulate enough current, the unit light is duplicated N_1 times, ensuring that all N_1 cross-points (indicated by the colored blocks with blue arrows in Figure 7) produce the same output. This configuration allows for the computation of $2(128 - N_1)$ elements parallelly. Finally, we employ the circuit shown on the right of Figure 7, which redirects and accumulates the currents from all N_1 cells into a single capacitor, producing the final output.

Multiplication: In the middle of Figure 7, we show an example of element-wise multiplication between two vectors, x and y . The vector x is modulated from the horizontal direction, while y is modulated from the vertical direction. For each input pair x_i and y_i , N_2 modulators are used repeatedly to accumulate the current, and the final readout is the summation from N_2^2 unit cells.

Scaling: Scaling involves multiplying an input matrix or vector, x , by a constant factor, a . As shown on the right of Figure 7, this operation resembles element-wise addition. We encode the coefficient a N_3 times in both directions and use the remaining $128 - N_3$ modulators to encode the input x , as illustrated in the figure. The results are obtained by summing the currents from N_3 unit cells.

The duplicate numbers N_1 , N_2 , and N_3 are determined using an optimal strategy to balance charge accumulation and parallelism. For a single cell, the charging duration is approximately 100 ns. For N duplicated cells, this duration is linearly reduced to $\frac{100}{N}$ ns (In the real-world scenario, it might need $> \frac{100}{N}$ ns due to the imperfection of the circuit, requiring a further recalibration). As the duplicate number N increases, parallelism is compromised. To maximize the number of element-wise operations within a fixed duration, e.g., 100 ns, the number of operations can be expressed as:

$$\text{Add: } (128 - N_1) \cdot N_1, \quad \text{Mul: } (128/N_2) \cdot N_2^2, \quad \text{Scl: } (128 - N_3) \cdot N_3$$

where $128 - N_1$ is the number of parallel inputs, and $N_1 = 100/(\frac{100}{N_1})$ is the number of operations for each cell that can be conducted in 100ns. We solve the optimal duplicate numbers to be $N_1, N_3 = 64$. This configuration enables 64 parallel inputs to be processed simultaneously, with a charging duration of 100/64 ns per cell. For multiplication, larger values of N_2 produce better results. To

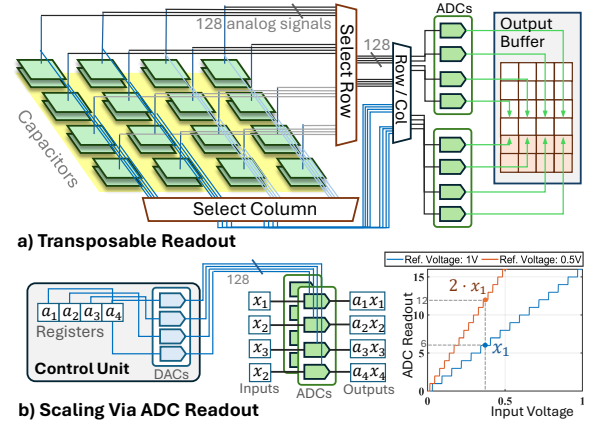


Figure 8: a) Circuit Design for Transposable Readout Module. b) Implementation for Scaling Unit Via ADCs.

ensure consistency with addition and compatibility with the same accumulation circuit, we select $N_2^2 = 64$, resulting in $N_2 = 8$.

6.4 Analog-Implemented Functions

ReLU Unit: The Rectified Linear Unit (ReLU) is a widely used activation function in machine learning, defined as $\text{ReLU}(x) = \max(0, x)$, which introduces non-linearity into models. It can be efficiently implemented in an analog circuit, as a diode-based analog filter can nullify negative values. In LightML, the ReLU function is applied before the ADC readout. As the analog signal is transferred from the capacitor to the ADC, a rectifier diode blocks any negative current, ensuring that only positive values are passed.

Transposable Readout: Matrix transpose is frequently used in AI models such as transformers, e.g., BERT, GPT, natural language processing, and recommendation systems. Supporting efficient transposition is challenging for DRAM-based memory, as it requires $O(n^2)$ non-sequential read/write operations, disrupting data locality [85]. We propose performing transposition at the ADC readout circuit, offering a faster and more cost-effective solution compared to existing methods like the transpose unit [25] or in-plane transpose [24].

After the optical crossbar completes the MAC operation, the results are stored in capacitors as analog signals, and our approach transposes the intermediate output when needed for the next layer. We illustrate the readout process in Figure 8 a). Each of the 128×128 results is connected to a port that carries the analog voltage from the capacitors. Each crosspoint result is connected to row and column selectors. When transposition is needed, the selectors route the inputs: row selectors to column ADCs and column selectors to row ADCs, effectively performing the transposition. While an ideal setup would use one ADC per crosspoint, this approach can cause significant power consumption, especially with high-resolution and fast-sampling ADCs. LightML features 8×128 ADCs operating at 0.9 GHz, and this process repeats for 16 rounds until all results are sent to the ADCs. The choice and number of ADCs are optimized to align with the throughput of the crossbar, output buffer, input buffer, and HBMs, details are in Section 8.4.

Scaling via ADCs: Section 6.3 introduces a general scaling method using the optical crossbar. We propose another cost-effective alternative using ADCs for scaling with a relatively small factor

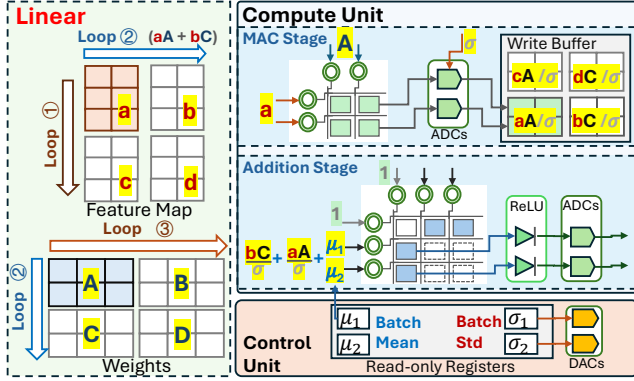


Figure 9: Implementation of Linear Transformation.

$a \in [0.25, 2]$, which is useful for frequently applied operations like batch normalization. An ADC converts continuous analog signals into discrete digital values, with the reference voltage determining the range of signals it can convert. This reference voltage, V_{ref} , typically represents the maximum voltage the ADC can measure and can be supplied externally or generated internally. The input voltage is mapped to a digital value based on its proportion to V_{ref} , following the formula: $\hat{x} = \frac{V_{in}}{V_{ref}} \times 2^n$ (2^n is the maximum digital value for an n -bit ADC). Thus, the digital output is essentially V_{in} divided by V_{ref} and digitized. This relationship can be used to implement a division by a coefficient a by setting $V_{ref} = a$, while the input is $V_{in} = x$. Note that V_{ref} functions effectively within the range $a \in [0.25, 2]$, considering the error-tolerant range of an 8-bit ADC [57]. Figure 8b) illustrates an example of scaling, along with the corresponding PSpice[54] simulation result. Here, the input V_{in} ranges from 0 to 1 V, with $V_{ref} = 0.5$ V. The ADC output accurately reflects the input scaled by a factor of 2, demonstrating the precision of this approach.

6.5 Linear Transformation

A linear transformation is a fundamental operation in AI models, involving the multiplication of an input feature map with a weight matrix. Linear layers form the backbone of machine learning models, and when combined with batch normalization and ReLU activations, they become core building blocks of deep learning and neural networks. A general linear layer applies a transformation to the input data and typically includes MMM/MVM, Batch Normalization, and Activation Function, which can be expressed as:

$$y = \text{ReLU}(Wx/\sigma - \mu')$$

where $x \in \mathbb{R}^{C_{in} \times b}$ is the input matrix/vector, e.g. feature map, with dimension C_{in} and b is the number of batches, $W \in \mathbb{R}^{C_{in} \times C_{out}}$ is the weight matrix, and μ and σ are the mean and standard deviation for batch normalization. Typically, the batch means and standard deviations are fixed during the inference [35] and are referred to as `running_mean` and `running_var` in Pytorch Platform [61].

Matrix Tiling: Building on existing MMM in photonic devices [26, 71, 89], LightML also introduces matrix tiling to handle large-scale MMM. The optical crossbar supports MMMs by, at maximum, two 128×1024 matrices (See Section 2.3), and tiling is necessary to accommodate MMMs with larger scales. Since LightML can

perform MMM for two 128×1024 matrices at one time, MMM for larger matrices would need tiling, as depicted in Figure 9. We tile the feature map into sub-matrices with 128 batches and 1024 in-features and the weights into sub-matrices with 1024 in-features and 128 out-features. In each iteration, we compute the multiplication between these two tiled sub-matrices. We iterate the computation in three loops: ① iterates over the batches. We sequentially select 128 batches from the feature map and multiply them with the same weight sub-matrix; ② iterates over the input features. The weight matrix slides 1,024 dimensions from left to right, while the feature map slides from top to bottom; ③ iterates over the output features.

Dataflow: In Figure 9, both the input feature map and weights are tiled into 128×1024 submatrices to fit into the crossbar. This process has two stages: In the MAC stage, the sub-matrices a , c , b , and d undergo the MAC operation with the sub-matrices A and C in the optical crossbar. The intermediate results, aA , bC , cA , and dC of 128×128 matrices, are stored in the output buffer. In the addition stage, we perform an element-wise addition of the intermediate matrices. This involves summing the intermediate results to obtain $aA + bC$ and $cA + dC$.

ReLU and BatchNorm: The Batch Normalization consists of two operations: dividing σ and subtracting μ , which can be performed through ADCs scaling, and then the element-wise addition. σ and μ are initially stored in the coefficient registers of the control unit. During the MAC stages, σ is converted into voltages as the reference voltages V_{ref} to read the MAC output. During the addition stage, we append the negative of the batch mean, $-\mu'_1, -\mu'_2, \dots, -\mu'_{C_{out}}$, as an extra term in the element-wise addition. Last, we use the analog ReLU unit to filter out negative values before ADC Readout.

6.6 Convolutional Operation

Convolution in machine learning is a critical component primarily for extracting features. The convolutional operation can be defined as $Y = W \otimes X$, where $X \in \mathbb{R}^{D \times D \times C_{in}}$ is the input feature map with width and height both D , and C_{in} is the number of input channels. $W \in \mathbb{R}^{K \times K \times C_{in} \times C_{out}}$ is a four-dimensional tensor representing the weights, where K is the kernel size and C_{out} is the number of output channels. $Y \in \mathbb{R}^{D \times D \times C_{out}}$ is the output feature map.

Challenges: Implementing the convolutional layer can be memory intensive due to the non-sequential distribution of convolution windows in memory. A common solution is the `im2col` operation, which unfolds the input feature maps into a matrix such that elements in each row correspond to one convolutional window. In classical computing, `im2col` is typically performed separately for each input channel, creating a $K^2 \times D^2$ matrix for each channel and then concatenating the C_{in} channels. This process requires loading each feature map into the on-chip cache and performing at least $K^2 \cdot D^2 \cdot C_{in}$ cache read/write operations.

Optimized Memory Structure: The proposed memory structure eliminates the cache-wise `im2col` operation. Instead, we utilize the transposable readout module, an on-chip address router, ensuring the computation unit and memory are efficiently utilized in most scenarios. A two-stack HBM2E contains 32 pseudo-channels and 8 banks for each, which can arbitrarily read 32KB of data within each bank. We distribute the $D \times D \times C_{in}$ feature map across 32 pseudo-channels, with each channel storing $\lceil C_{in}/32 \rceil$ of $D \times D$ matrices.

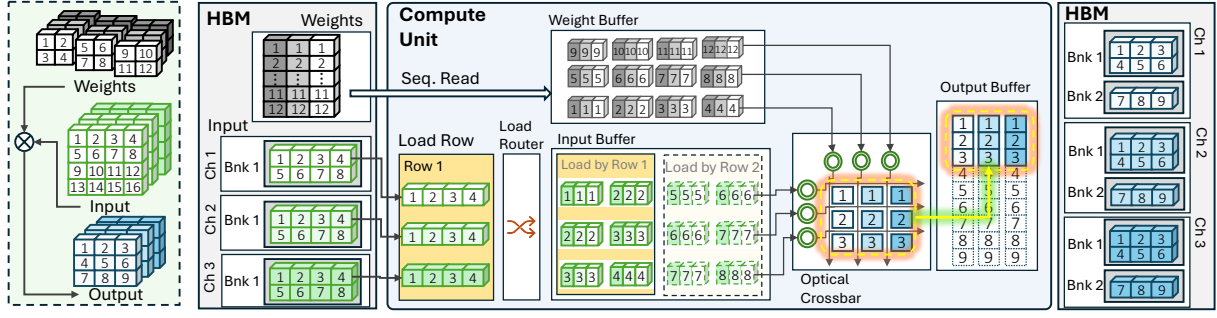


Figure 10: Implementation of Convolutional Operation.

We group the elements in the same location for C_{in} channels into a line, and the `im2col` operation applies the same operation to the line rather than a single element. Our approach transposes the feature map and the weight into a matrix, where we span the $D \times D$ feature maps into D^2 columns, and all C_{in} columns are concatenated into a matrix (it can be achieved by reading out from the row selector). The weight is also converted into a $K^2 C_{out} \times C_{in}$ matrix (prepared during the offline period).

Figure 10 shows an example of a $4 \times 4 \times 3$ feature map being convolved with $2 \times 2 \times 3 \times 3$ weights. We use digits to annotate the locations of elements in the feature map and different colors to represent different channels. In the example, the three-channel feature map is distributed across three pseudo-channels in the HBM, with their first bank, Bnk1, stores a 2×4 matrix, i.e., 1234-5678. During memory loading, each channel loads a 1×4 matrix to the load row on the chip, forming a data line as 1234-1234-1234. Next, we use the load router to reorder the data line into three lines: 111-222, 222-333, and 333-444 and write these lines into the input buffer. By repeating the memory loading process twice, we prepare the data for convolution. We then feed the unfolded feature map and weights (weights are preprocessed and sequentially read into the weights buffer) into the optical crossbar for the MAC operation. The MAC result is a 3×3 matrix. We use the transposable readout to read the mesh in a column-wise order, distributing results for different output channels into different buffer lines. The same data structure stores the results in the HBM, which will serve as the input feature map for the next layer.

Tiling: The tiling for the convolutional layer is similar to the linear layer, involving three loops: ①: As the optical crossbar has 128 ports, we tile a $D \times D$ feature map by selecting $n = \lfloor 128/D \rfloor$ rows to form sub-matrices of no more than 128 elements. If D exceeds 128, we split each row into $\lceil D/128 \rceil$ 128-dimension chunks. For each iteration, besides the n target rows, we fetch $(K-1)$ additional rows to the input buffer as the top and bottom rows required for the convolution. This loop repeats $\lceil D/n \rceil$ times. ②: Given the limit of 1,024 MAC operations, each iteration accommodates up to $\lceil 1024/K^2 \rceil$ input channels, forming no more than 1,024 MAC operations. This loop repeats $\lceil C_{in} \times K^2 / 1024 \rceil$ times. ③: The final loop iterates over the output channels, selecting 128 output channels each time and repeating for $\lceil C_{out}/128 \rceil$ rounds.

7 Pipelining and Scheduling

To ensure all hardware units remain fully utilized, the computing speed of the crossbar must be aligned with the data transfer rates to

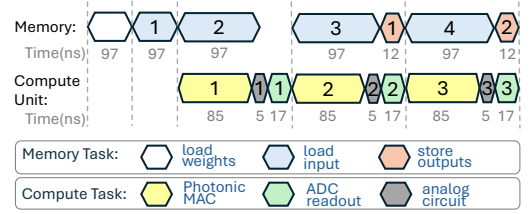


Figure 11: Example of Scheduling for Linear Transformation.

and from the input and output buffers. A pipeline illustrating memory loads, stores, computation, analog circuit, and ADC readout is shown in Figure 11, where we capture a scheduling example of a MMM operation tiled into 4 submatrices. In the computing unit, the crossbar's latency is proportional to the length of the dot-product operation, determined by the number of pulses modulated at each crosspoint. For a modulator operating at 12 GHz and a crossbar performing 1,024 continuous pulses, each dot-product operation takes approximately 85 ns. The analog circuit, including the ReLU unit, requires 5 ns to stabilize its output (Simulated Via PSpice). Additionally, the system incorporates 8×128 ADCs operating at 0.9 GHz, which need 16 rounds (17.7 ns) to complete the readout. On the memory side, feature maps are transmitted from HBM modules to the input buffers. A double-buffer scheme allows one buffer to receive data from the HBM while the other feeds the modulator. Since SRAM buffers are significantly faster than HBM, the bottleneck lies in the HBM read latency. For a sequential read of a 128×1024 matrix, the latency is approximately 97 ns. Subsequently, the results from the output buffer, a 128×128 matrix, are written back to the HBMs, requiring around 12 ns.

8 Evaluations

8.1 Experiment Setup

Model and Dataset: In this work, we evaluate the performance of LightML across a diverse set of CNNs and datasets. For the dataset, we use CIFAR-10 [40], CIFAR-100 [40], and ImageNet [14]. The evaluated models cover a spectrum of sizes, we use ResNet-18 (11M parameters), ResNet-50 (25M), ResNet-101 (45M) [29], VGG-11 (133M), VGG-16 (138M), VGG-19 (144M) [70], MobileNet-V2 (3.5M), MobileNet-V3-Large (5.4M) [32]. We choose 32 as the batch size for the inference images, where the A100 GPU reaches its maximum efficiency.

Simulation and Platform: We implement LightML in a cycle-accurate simulator from scratch. We implement the modules for

Table 3: Hardware Configuration of LightML.

Component	Size / #	Freq.(Hz)	Power	Area(mm ²)
On-Chip Buffer	256/128/64KB	2G	84mW	0.68
ADCs [57]	128x8	0.9G	1.93W	5.73
Capacitors	128x128x2	-	-	-
Laser Source[7]	1	-	120mW	-
Modulators [84]	128x2+128	12G	810mW	69.1
Detectors[55]	128x128x2	12G	21.8mW	235
On-Chip Total	-	-	2.97W	310
HBM [42]	2x16GB	1.6G	~2x8W	~2x80
Total	-	-	~19W	~470

each proposed built-in function unit in the simulator. The simulator includes the memory module, the on-chip buffer, and the optical crossbar. The analog circuit, comprising the RC charging circuit and ADC readout, is simulated using PSpice [54], while the power and area of the on-chip buffers are estimated with CACTI [4].

For the baseline comparison, latency results are measured on a real machine running the PyTorch platform [61]. The hardware setup includes an Intel i9-13900K for CPU, an NVIDIA A100 [9, 10] for GPU, and a Google TPU V3 [37]. GPU and TPU measurements use FP16 data precision, and CPU uses Int8 (Int8 and Int4 are not widely adopted in CUDA or XLA environments. The precision configuration is aligned with existing crossbar work [28, 34, 72]).

The implementation and experimental setup are available at: <https://github.com/Liang78825/LightML.git>.

8.2 Specification Detail

Optical Layout: In our layout, we have assumed that each crossbar unit cell is $120\mu\text{m} \times 120\mu\text{m}$ [89]. This space includes the two directional couplers, 3 dB splitter, and on-chip balanced Ge photodetectors. A recent work by Rogers et al. [64] demonstrated a functional 512-pixel LiDAR receiver array on a silicon photonic platform that contained one directional coupler, 3 dB splitter, balanced Ge photodetectors, and an optical antenna in an $80\mu\text{m} \times 100\mu\text{m}$ unit cell. Notably, in this example, the optical antenna (excluded from our platform) comprised the majority of the layout space rather than the other optical components.

Electrical Devices: The RC charging circuit was designed using PSpice simulations to minimize charging duration while accumulating optical current to 1V with minimal noise. The circuit employs two optical-electrical transceivers [60], producing currents from -600 nA to 600 nA. It incorporates a 15 fF capacitor (similar to DDR4 technology), a 500 Ω resistor, and an 8-bit SAR-ADC operating at 0.9 GHz[57]. Through the simulation result of the RC circuit, the

circuit’s error distribution is characterized by $\mathcal{N}(0.00037, 0.0024)$ after 1,024 pulse accumulations.

Table 3 summarizes the hardware configuration of LightML. The on-chip buffer consists of three double buffers: an input buffer (256KB), a weight buffer (128KB), and an output buffer (64KB), operating at a frequency of 2 GHz. LightML adopts 8 sets of 128 ADCs. Each crosspoint in the 128×128 array contains 2 capacitors, and its area is included in the Detector modules. The laser source[7] is integrated externally, operating at a wavelength of 1550nm, with adjustable power consumption up to 150mW. The modulators[84] are arranged in 128×2 for amplitude and 128 for phase. The modulator frequency can be scaled up to 50GHz, and we choose 12GHz to match the memory throughput. The homodyne detection scheme requires 2 photodetectors in every crosspoint of the 128×128 crossbar. Overall, LightML has a total power consumption of 2.97W and occupies an area of 310 mm². In addition to considering on-chip integrated HBM, we also investigate Micron’s HBM2E [75]. The power consumption of HBM2E varies with temperature, supply voltage, and workload. For our analysis, we assume a supply voltage of 1V under full load at room temperature, which results in a power consumption of 8W per stack as reported in the technical documentation[42]. Moreover, based on data from the NVIDIA A100 [9], which utilizes an 8-stack HBM2E configuration—we estimate the package area to be approximately 80 mm².

8.3 Comparison with Existing Accelerators:

To the best of our knowledge, this is the first study to provide architectural support for photonic crossbars. Some previous studies have performed high-level evaluations but with limited architectural design (details in Section 10). In this work, we deliver a comparison with general-purpose ML processing via CPU, GPU, and TPU, and classical PE-based accelerator (ThinkFast[1]), PE-based PIM (SP-PIM [39]), and ReRAM-based crossbar RRAM-CIM[80] and RRAM-CIM2 [87]. The comparison includes performance efficiency (PE) and computational efficiency (CE).

Power consumption is a critical factor for scalability and deployment in edge devices. GPUs, TPUs, and ThinkFast consume 210W, 185W, and 300W, whereas LightML operates at only 2.97W. Note that, because the memory architectures differ across platforms, the reported power values exclude the memory subsystem. PE and CE highlight the benefits of LightML. With a PE of 109 TOP/s/W and a CE of 1.04 TOP/s/mm², LightML outperforms GPU by $109/1.48 = 73.6\times$ and the state-of-the-art NVM-based accelerators by more than $105/57 = 1.91\times$ in terms of efficiency. The comparison indicates that LightML offers a compelling combination of

Table 2: Comparison of Various AI Accelerators Across Different Specs.

Platform	GPU	TPU	ThinkFast[1]	SP-PIM[39]	RRAM-CIM[80]	RRAM-CIM2[87]	LightML
Technology	7nm	12nm	14nm	28nm	130nm	40nm	28nm
Frequency	765MHz	940MHz	0.9GHz	450MHz	-	100MHz	12GHz
Precision	FP16	FP16	Int8	FP16	Int2/4	Int2/4	Int5
Peak Perf.	312TFLOP/s	180TFLOP/s	820TOP/s	3.2TOP/s	1.8TOP/s	0.63TOP/s	325TOP/s
Area	826mm ²	648mm ²	725mm ²	5.75mm ²	159mm ²	0.4mm ²	310mm ²
Power	~210W	~185W	300W	0.14W	45mW	11mW	2.97W
PE (TOP/s/W)	1.48	0.97	2.73	22.4	40	57	109
CE (TOP/s/mm ²)	0.37	0.27	1.13	0.55	0.011	1.57	1.04

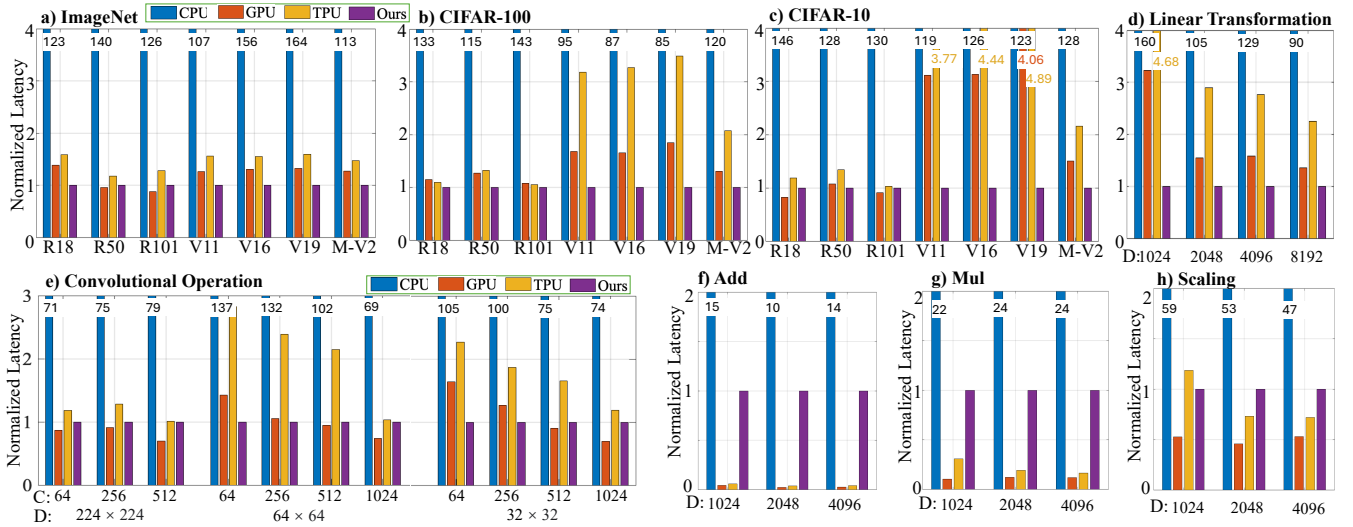


Figure 12: (a), (b), and (c) show the normalized inference latency for ResNet18, ResNet50, ResNet101 (R18, R50, R101), VGG11, VGG16, VGG19 (V11, V16, V19), and MobileNet-V2 (M-V2) with 32-batch input images from ImageNet, CIFAR-100, and CIFAR-10; (e) present the latency for a $D \times D \times C \times 32$ feature map convolving with a $3 \times 3 \times C \times C$ kernel; (d), (f), (g), and (h) show the latency results for $D \times D \times 32$ inputs.

high performance, low power consumption, and efficient utilization of area, making it a suitable candidate for various ML applications, particularly in power-constrained environments. If the power consumption of memory components (i.e., HBMs) is taken into account, then LightML outperforms the GPU by a factor of $(325/19W)/(312/250W)=13.6\times$.

8.4 Sensitive Study – Number of ADCs

An ADC typically has long latency and consumes a non-negligible amount of power. If every crosspoint in the crossbar is equipped with its own ADC, the total ADC power would dominate the overall power consumption, thereby reducing the power efficiency (PE). As an alternative, a single ADC can be multiplexed across multiple crosspoints to lower the total ADC power—for example, a single row of ADCs can serve several rows of the crossbar. Thus, the ADC configuration plays a critical role in balancing PE and performance.

To evaluate this tradeoff, we vary the number of ADC rows multiplexed across the entire crossbar: 1x128, 2x128, 4x128, 8x128, and 16x128. The results are summarized in Table 4. As expected, power consumption increases with the number of ADCs, ranging from 1.27W to 4.88W. Peak performance also improves with more ADCs, increasing from 147TOP/s to 356TOP/s. PE reaches its maximum at the 4x128 configuration.

Table 4: Sensitive Study of ADC Configuration.

ADC Conf.		1x128	2x128	4x128	8x128	16x128
Power (W)		1.27	1.51	1.99	2.97	4.88
Perf. (TOP/s)		147	214	277	325	356
PE (TOP/s/W)		115	141	138	109	72
R18	Latency	9.4ms	6.3ms	4.8ms	4.0ms	3.7ms
	Image/W/s	2686	3351	3348	2670	1792
R50	Latency	39ms	23ms	16ms	13ms	11ms
	Image/W/s	641	889	975	822	563
R101	Latency	63ms	39ms	27ms	23ms	20ms
	Image/W/s	397	538	577	466	314

Additionally, we evaluate the latency of model inferences with respect to the ADC configuration, specifically for ResNet18, 50, and 101 running on ImageNet with a batch size of 32. The latency and energy efficiency (measured as the number of images processed per Watt per second) are detailed in Table 4. The results show that the 4x128 configuration not only provides the best PE but also demonstrates optimal efficiency for processing images. Overall, these findings highlight the importance of selecting an appropriate number of ADCs to optimize the balance between power consumption and performance in LightML. Notably, the 8x128 setup offers an 17% performance gain at the cost of only 1W additional power, making it a worthwhile tradeoff.

8.5 Latency Comparison across ML Models/Ops

In this section, we compare the latency of LightML with that of CPU, GPU, and TPU. Latency data for CPU, GPU, and TPU is collected from real machines, the latency for the baseline machine is measured after the model and data are loaded to the device memory (`model.to(device)`), and we repeat the inference for 150 rounds and average the results for the last 100 rounds. GPU and TPU devices are synchronized before the start of each round. We use a batch size of 32 for all latency evaluations. We evaluate the normalized latency by dividing the actual latency of each device by the latency of LightML.

Figures 12 f), 12 g), and 12 h) assess the normalized latency of Addition, Multiplication, and Scaling. The results show that LightML is not optimal for element-wise operations due to poor crossbar utilization, with at most 1/64 of the crossbar being used. Specifically, LightML is 8.2x to 9.7x slower than the NVIDIA A100 for Multiplication and 1.9x to 2.1x slower for Scaling.

Figures 12d) and 12e) evaluate the latency of the Linear and Convolutional (Conv) layers. The evaluation for convolution and linear layers integrates ReLU and Batch Normalization. The results show that LightML outperforms GPU by more than 1.4x for the linear layer, with a performance gain of up to 3.2x for smaller feature

Table 5: Comparison of Existing Non-Linear Function Units.

NFU	Bits	Delay	Extra Power	Extra Area	Supported Functions
Zamanlogy [91]	6	2.12ns	64mW	0.27mm ²	σ
PLAN[59]	8	3.71ns	57mW	-	σ
RALUT [43]	8	2.12ns	-	1.06mm ²	σ , tanh
Optim-LUT [49]	8	2.82ns	590mW	0.78mm ²	σ , tanh
PWL LUT [78]	8	0.95ns	466mW	0.22mm ²	σ , tanh
LightML	5	3.1ns	N/A	0	Any

maps. For the convolutional layer, LightML slightly outperforms the GPU for 64×64 and 32×32 inputs, with performance gains ranging from 0.7x to 1.6x.

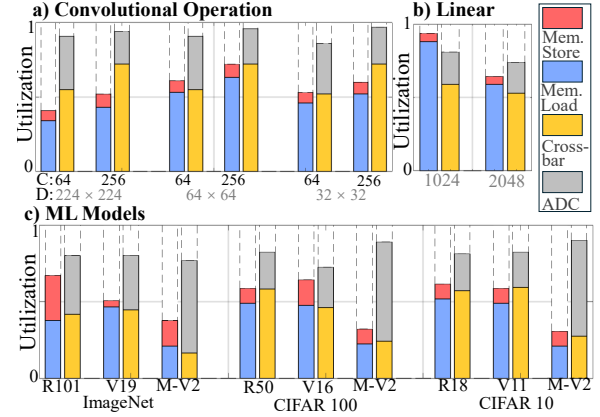
Figures 12 a), b), and c) present the normalized latency results for ImageNet, CIFAR-100, and CIFAR-10, respectively. For ResNet models, the latency of LightML is approximately 0.9x to 1.3x compared to GPU. For VGG models, LightML outperforms the GPU with performance gains of 1.3x to 4.0x.

As shown in the results, LightML provides promising performance compared to GPU and TPU devices, operating at a power consumption of only 3W. However, there are two main drawbacks: 1. The crossbar dimension (128×128) is small compared to the GPU and TPU, where the A100 has 108×256 tensor cores, each delivering 4 FP16 operations per core [9, 10]. TPU v3 comprises four Matrix-Multiplication Units, each performing MMM for $128 \times 128 \times 2$ input [37]. 2. LightML is not well-suited for element-wise operations due to low crosspoint utilization, a limitation shared by NVM-based crossbars. Future optimizations could consider incorporating a dedicated optical unit specifically designed for element-wise operations.

8.6 Comparison of Non-Linear Function Units

Prior Non-linear Function Unit (NFU) in ML accelerators generally falls into two categories. The first uses modular approximations (e.g., Piecewise Linear (PWL)) [59, 91], which segment nonlinear functions into linear pieces but have limited applicability (e.g., sigmoid only). The second uses Look-Up Tables (LUTs) to support a broader range of functions at the cost of higher area and power. Other methods, such as Taylor expansions, are generally too costly. In contrast, LightML computes nonlinear functions using two rounds of ADC readouts. Similar to element-wise operations, its NFU supports $(128 - 20) \times 2 = 216$ inputs concurrently: 108 vertical modulators generate the phase and 20 horizontal modulators for amplitude, with a similar configuration in the other direction. No extra hardware is required—our optical modulators naturally generate sine and cosine functions, enabling arbitrary nonlinear function implementation.

In Table 5, we compare the NFU of LightML against prior methods. Since the NFU in LightML can process 216 inputs per round, we use the same number of NFU units in the baseline for a fair comparison. Compared to existing approaches, LightML offers two key advantages: 1. It leverages existing hardware without requiring additional circuits, resulting in negligible power and area overhead. This simplifies fabrication and eliminates the need for data movement across different function units. 2. The optical NFU can implement a broad range of nonlinear functions using the same hardware, which is hard to achieve in prior methods.

**Figure 13: Utilization of Memory / Computing Unit.**

8.7 Utilization Analysis of LightML

In this section, we analyze the utilization of three key components of LightML: memory load/store, optical MAC, and ADC readout. As discussed in Section 7, the utilization encompasses two parallel processes: (1) involves the memory process and (2) involves the computing unit.

The utilization results are depicted in Figure 13, and the notations and configuration are the same as Figure 12. The color bar shows the breakdown latency of each operation. For example, when we measure the individual latency of each process, and the utilization is calculated by dividing the component's latency by the overall latency. For memory operations, we display only the load and store latency, excluding the latency due to page faults and row activation, which are included in the overall latency but not shown in the figure. Figure 13 a) illustrates the utilization when computing the convolutional layer. For the convolution layer, the memory process shows under-saturation, with utilization rates between 40% and 60%. Conversely, the computing unit achieves more than 90% utilization.

8.8 Impact of Precision on Model Accuracy

In this section, we evaluate the accuracy of ML models using quantization sampled from crossbar devices. We simulate $N_b = 5$ and $N_b = 8$ modes by quantizing the inputs and outputs of each MAC operation to 5 and 8 bits, respectively, and applying the same Gaussian noise as described in Section 3.2. In 5-bit mode, the error rate is low (0.0034), while in 8-bit mode, the error rate reaches 0.632; however, the mean error distance is lower in the 8-bit mode. For simulating the NVM-based crossbar, we use 4-bit precision to encode the intermediate map and ($N_b = 2$ or $N_b = 3$)-bits for the weights. We assume separate functional units for addition and non-linear functions, providing 8-bit precision. All noise and precision loss in the simulation are generated using Gaussian noise.

The results, presented in Table 6, show the accuracy of different models under varying precision levels. The simulations were conducted on the PyTorch platform using a three-layer convolutional network for MNIST [15], ResNet-18 for CIFAR-10, and MobileNetV2 for ImageNet. For MNIST, LightML matches the GPU/TPU performance, reaching 99.2% accuracy. For CIFAR-10 and ImageNet, LightML is slightly lower than the accuracy of GPU/TPU but significantly higher than the NVM-based crossbars.

Table 6: Accuracy under Different Precision Models.

Dataset	MNIST	CIFAR-10	ImageNet
NVM ($N_b = 2$)	98.1%	84.8%	60.3%
NVM ($N_b = 3$)	99.0%	86.4%	63.5%
LightML($N_b = 5$)	99.2%	90.6%	66.1%
LightML($N_b = 8$)	99.1%	91.1%	66.5%
GPU/TPU (FP16)	99.2%	92.4%	69.8%

9 Feasibility for Large Language Models

Large Language Models (LLMs) [6, 16, 79] are widely used in machine learning but demand substantial computation and energy for training and inference. Recent studies show they tolerate lower precision, with only a 1–2% accuracy drop using FP8 and FP4 [23], making them viable for LightML deployment.

An algorithm example of an attention block is shown below. This block utilizes two LightML functions: LightML.Linear, which performs the linear transformation to compute Q , K , and V matrices; and LightML.NonLinear, which computes the softmax via a non-linear function. Additionally, memory operations, such as head splitting and token padding, are simulated using a basic memory read/write strategy.

Algorithm 1 Implementation for Attention Block

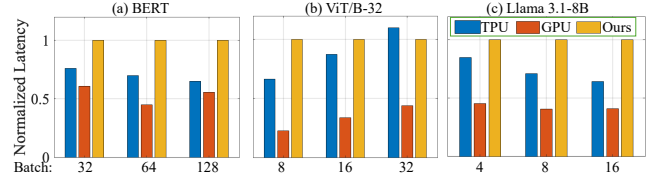
```

1:  $x \leftarrow$  Input Token
2:  $xq = \text{LightML.Linear}(x, W_q)$ 
3:  $xk = \text{LightML.Linear}(x, W_k, \text{transpose}=\text{True})$ 
4:  $xv = \text{LightML.Linear}(x, W_v)$ 
5:  $\text{queries}[:, :] = \text{Memory.Split\_Head}(xq, \text{num\_head})$ 
6:  $\text{keys}[:, :] = \text{Memory.Split\_Head}(xk, \text{num\_head})$ 
7: for  $i$  in range( $\text{num\_head}$ ) do
8:    $\text{scores}[i] = \text{LightML.Linear}(\text{queries}[i], \text{keys}[i], \text{scale} = \sqrt{\text{head\_dim}})$ 
9: end for
10:  $\text{scores} = \text{LightML.NonLinear}(\text{scores}, \text{'softmax'})$ 
11:  $\text{scores} = \text{Memory.Combine\_head}(\text{scores}, \text{num\_head})$ 
12:  $\text{output} = \text{LightML.Linear}(\text{scores}, \text{values})$ 

```

We present a preliminary implementation of BERT[16], Llama 3.1 8B[77], and ViT/B-16[17], normalized to the performance of LightML as shown in Figure 14. We compare against two baselines—Google TPU and NVIDIA A100 GPU—both tested on real machines, with the GPU using INT8 quantization. For BERT and Llama, tokenization is assumed to run on the CPU, processing input sequences of class digits. Sequences start with eight tokens, with model completion up to 128 tokens. For the ViT, inputs are sampled from ImageNet with 384×384 pixel images and 1,000 classes.

Our current implementations lack proper optimizations for LLM models, resulting in performance disadvantages compared to the two baselines. In the Llama model, an NVIDIA A100 GPU is about 2.2x faster than LightML, but LightML achieves 6x higher energy efficiency per token. A detailed analysis of the attention block reveals two key inefficiencies. First, attention inputs, represented by the Q and K matrices of size $N_{\text{tokens}} \times D$, often have fewer token counts (N_{tokens}) than the crossbar dimension (128) for short sequences, leading to underutilization. Second, element-wise operations (e.g., addition) on large matrices are computationally inefficient on the crossbar, contributing 20% to attention overhead. Overall, while our results highlight the potential of photonic circuits for LLMs, further optimizations are needed for future work.

**Figure 14: LLM Inference Latency.**

10 Related Work

Hamerly et al. [26] propose an optical architecture using homodyne detection to perform MAC operations. The system relies on an electronic controller, such as a CPU, to generate input signals and handle nonlinear functions. By reformulating convolutions as matrix-matrix multiplications, they demonstrate support for both fully connected and convolutional layers. The design achieves 100aJ per MAC (50mW) on a 100×100 crossbar, excluding power for memory, ADCs, and the laser source.

Sludds et al. [71] present Netcast, an optical edge-computing architecture where model weights are encoded via wavelength-division multiplexing (WDM) and transmitted as light from the cloud to an on-device crossbar for processing. Demonstrated on a two-layer MLP for MNIST, Netcast achieves energy consumption as low as 40aJ per operation (40mW).

Li et al. [45] present a scalable accelerator that leverages a modular design where each chiplet contains processing elements interconnected using optical waveguides.

Shiflett et al. [69] propose Flumen, a dual-function photonic interconnect that enables both data communication and in-network computation. The architecture utilizes a Mach-Zehnder interferometer mesh to perform MMM directly on optical signals through an SVD-based transformation. This design allows dynamic switching between communication and computation.

Giamougiannis et al. [22] design a coherent photonic crossbar (Xbar) that maps matrix elements to dedicated optical nodes. Rather than decomposing a matrix via SVD, the Xbar modulates the input signal with programmable amplitude and phase at each node, enabling scalable linear transformations with linear loss dependence.

To the best of our knowledge, LightML is the first architecture to support an optical computing device, efficiently handling high-bandwidth optical signals with pipelined model execution and optimized power and compute efficiency. It also pioneers the implementation of general nonlinear functions within the optical crossbar—a key capability unmet by prior opto-electronic accelerators and crucial for modern AI models.

11 Conclusion

We presented LightML, a high-performance photonic computing architecture optimized for ML applications. Supporting diverse ML operations, LightML is ideal for edge devices and data centers. It offers superior computing speed and energy efficiency, outperforming state-of-the-art solutions.

Acknowledgments

The authors would like to thank the anonymous reviewers for their valuable feedback and suggestions. This work is supported by NSF grants #2334628, #2337674, #2154973, as well as AFOSR grant FA9550-24-1-0064.

References

- [1] Dennis Abts, Jonathan Ross, Jonathan Sparling, Mark Wong-VanHaren, Max Baker, Tom Hawkins, Andrew Bell, John Thompson, Temesghen Kahsai, Garrin Kimmell, et al. 2020. Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 145–158.
- [2] Shuroug A Alowais, Sahar S Alghamdi, Nada Alsuhebany, Tariq Alqahtani, Abdulrahman I Alshaya, Sumaya N Almohareb, Atheer Aldairem, Mohammed Alrashed, Khalid Bin Saleh, Hisham A Badreldin, et al. 2023. Revolutionizing healthcare: the role of artificial intelligence in clinical practice. *BMC Medical Education* 23, 1 (2023), 689.
- [3] Farshid Ashtiani, Alexander J. Geers, and Firooz Aflatouni. 2022. An on-chip photonic deep neural network for image classification. *Nature* 606 (6 2022), 501–506. Issue 7914. <https://doi.org/10.1038/s41586-022-04714-0>
- [4] Rajeev Balasubramanian, Andrew B Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New tools for interconnect exploration in innovative off-chip memories. *ACM Transactions on Architecture and Code Optimization (TACO)* 14, 2 (2017), 1–25.
- [5] Colby R Banbury, Vijay Janapa Reddi, Max Lam, William Fu, Amin Fazel, Jeremy Holleman, Xinyuan Huang, Robert Hurtado, David Kanter, Anton Lokhmotov, et al. 2020. Benchmarking tinyml systems: Challenges and direction. *arXiv preprint arXiv:2003.04821* (2020).
- [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. *arXiv preprint arXiv:2005.14165* (2020).
- [7] Bigeng Chen, Xingshi Yu, Xia Chen, Milan M Milosevic, David J Thomson, Ali Z Khokhar, Shinichi Saito, Otto L Muskens, and Graham T Reed. 2018. Real-time monitoring and gradient feedback enable accurate trimming of ion-implanted silicon photonic devices. *Optics express* 26, 19 (2018), 24953–24963.
- [8] Bill Corcoran, Mengxi Tan, Xingyuan Xu, Andreas Boes, Jiayang Wu, Thach G. Nguyen, Sai T. Chu, Brent E. Little, Roberto Morandotti, Arnan Mitchell, and David J. Moss. 2020. Ultra-dense optical data transmission over standard fibre with a single chip source. *Nature Communications* 11 (5 2020), 2568. Issue 1. <https://doi.org/10.1038/s41467-020-16265-x>
- [9] NVIDIA Corporation. 2020. NVIDIA A100 Tensor Core GPU Architecture Whitepaper. <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf> Accessed: 2024-06-11.
- [10] NVIDIA Corporation. 2020. NVIDIA A100 Tensor Core GPU Datasheet. <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf> Accessed: 2024-06-11.
- [11] Ivan Cvitić, Dragan Peraković, Marko Periša, and Brij Gupta. 2021. Ensemble machine learning approach for classification of IoT devices in smart home. *International Journal of Machine Learning and Cybernetics* 12, 11 (2021), 3179–3202.
- [12] Cansu Demirkiran, Furkan Eris, Gongyu Wang, Jonathan Elmhurst, Nick Moore, Nicholas C Harris, Ayon Basumallik, Vijay Janapa Reddi, Ajay Joshi, and Darius Bunandar. 2023. An electro-photonic system for accelerating deep neural networks. *ACM Journal on Emerging Technologies in Computing Systems* 19, 4 (2023), 1–31.
- [13] Cansu Demirkiran, Guowei Yang, Darius Bunandar, and Ajay Joshi. 2024. Mirage: An RNS-based photonic accelerator for DNN training. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 73–87.
- [14] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 248–255.
- [15] Li Deng. 2012. The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine* 29, 6 (2012), 141–142.
- [16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [17] Alexey Dosovitskiy. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [18] Johannes Feldmann, Nathan Youngblood, Maxim Karpov, Helge Gehring, Xuan Li, M. Stappers, M. Le Gallo, Xin Fu, Anton Lukashchuk, Arslan S. Raja, Junqiu Liu, C. D. Wright, Abu Sebastian, Tobias J. Kippenberg, W. H. P. Pernice, and Harish Bhaskaran. 2021. Parallel convolutional processing using an integrated photonic tensor core. *Nature* 589 (1 2021), 52–58. Issue 7840. <https://doi.org/10.1038/s41586-020-03070-1>
- [19] Manuel Le Gallo, Riduan Khaddam-Aljameh, Milos Stanisavljevic, Athanasios Vasileopoulos, Benedikt Kersting, Martino Dazzi, Geethan Karunaratne, Matthias Brändli, Abhairaj Singh, Silvia M. Müller, Julian Büchel, Xavier Timonedá, Vinay Joshi, Malte J. Rasch, Urs Egger, Angelo Garofalo, Anastasios Petropoulos, Theodore Antonakopoulos, Kevin Brew, Samuel Choi, Injo Ok, Timothy Philip, Victor Chan, Claire Silvestre, Ishfaq Ahsan, Nicole Saulnier, Vijay Narayanan, Pier Andrea Francesc, Evangelos Eleftheriou, and Abu Sebastian. 2023. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference. *Nature Electronics* 6 (8 2023), 680–693. Issue 9. <https://doi.org/10.1038/s41928-023-01010-1>
- [20] Eva Garcia-Martin, Crefeda Faviola Rodrigues, Graham Riley, and Håkan Grahn. 2019. Estimation of energy consumption in machine learning. *J. Parallel and Distrib. Comput.* 134 (2019), 75–88.
- [21] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. 2022. A survey of quantization methods for efficient neural network inference. In *Low-Power Computer Vision*. Chapman and Hall/CRC, 291–326.
- [22] George Giamougiannis, Apostolos Tsakyridis, Yangjin Ma, Angelina Totović, Miltiadis Moralis-Pegios, David Lazovsky, and Nikos Pleros. 2023. A coherent photonic crossbar for scalable universal linear optics. *Journal of Lightwave Technology* 41, 8 (2023), 2425–2442.
- [23] Zhuocheng Gong, Jiahao Liu, Jingang Wang, Xunliang Cai, Dongyan Zhao, and Rui Yan. 2024. What Makes Quantization for Large Language Model Hard? An Empirical Study from the Lens of Perturbation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 18082–18089.
- [24] Fred G Gustavson and Tadeusz Swirszcz. 2007. In-place transposition of rectangular matrices. In *Applied Parallel Computing. State of the Art in Scientific Computing: 8th International Workshop, PARA 2006, Umeå, Sweden, June 18-21, 2006, Revised Selected Papers 8*. Springer, 560–569.
- [25] Nastaran Hajinazar, Geraldo F Oliveira, Sven Gregorio, João Ferreira, Nika Mansouri Ghiasi, Minesh Patel, Mohammed Alser, Saugata Ghose, Juan Gómez Luna, and Onur Mutlu. 2021. SIMDram: An end-to-end framework for bit-serial SIMD computing in DRAM. *arXiv preprint arXiv:2105.12839* (2021).
- [26] Ryan Hamerly, Liane Bernstein, Alexander Sludss, Marin Soljačić, and Dirk Englund. 2019. Large-scale optical neural networks based on photoelectric multiplication. *Physical Review X* 9, 2 (2019), 021032.
- [27] Ryan Hamerly, Liane Bernstein, Alexander Sludss, Marin Soljačić, and Dirk Englund. 2019. Large-Scale Optical Neural Networks Based on Photoelectric Multiplication. *Physical Review X* 9 (5 2019), 021032. Issue 2. <https://doi.org/10.1103/PhysRevX.9.021032>
- [28] Muhammad Abdullah Hanif, Aditya Manglik, and Muhammad Shafique. 2020. Resistive crossbar-aware neural network design and optimization. *IEEE Access* 8 (2020), 229066–229085.
- [29] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [30] S Hochreiter. 1997. Long Short-term Memory. *Neural Computation MIT-Press* (1997).
- [31] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. 2024. Training compute-optimal large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (New Orleans, LA, USA.) (NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 2176, 15 pages.
- [32] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. MobileNet: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).
- [33] Junying Huang, Rongliang Fu, Xiaochun Ye, and Dongrui Fan. 2022. A survey on superconducting computing technology: circuits, architectures and design tools. *CCF Transactions on High Performance Computing* 4, 1 (2022), 1–22.
- [34] Ruirong Huang, Zichao Yue, Caroline Huang, Janarbek Matai, and Zhiru Zhang. 2023. Comprehensive Benchmarking of Binary Neural Networks on NVM Crossbar Architectures. *arXiv preprint arXiv:2308.06227* (2023).
- [35] Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*. pmlr, 448–456.
- [36] Hasitha Jayatileka, Harel Frish, Ranjeet Kumar, John Heck, Chaoxuan Ma, Meer N Sakib, Duanni Huang, and Haisheng Rong. 2021. Post-fabrication trimming of silicon photonic ring resonators at wafer-scale. *Journal of Lightwave Technology* 39, 15 (2021), 5083–5088.
- [37] Norman P Jouppi, Doe Hyun Yoon, George Kurian, Sheng Li, Nishant Patil, James Laudon, Cliff Young, and David Patterson. 2020. A domain-specific supercomputer for training deep neural networks. *Commun. ACM* 63, 7 (2020), 67–78.
- [38] Sadra Rahimi Kari, Nicholas A. Nobile, Dominique Pantin, Vivswan Shah, and Nathan Youngblood. 2024. Realization of an integrated coherent photonic platform for scalable matrix operations. *Optica* 11 (4 2024), 542. Issue 4. <https://doi.org/10.1364/OPTICA.507525>
- [39] Jung-Hoon Kim, Jaehoon Heo, Wontak Han, Jaekuk Kim, and Joo-Young Kim. 2023. SP-PIM: A 22.41 TFLOPS/W, 8.81 Epochs/Sec Super-Pipelined Processing-In-Memory Accelerator with Local Error Prediction for On-Device Learning. In *2023 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. IEEE, 1–2.
- [40] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).

- [41] Rolf Landauer. 1989. Johnson-Nyquist noise derived from quantum mechanical transmission. *Physica D: Nonlinear Phenomena* 38, 1–3 (1989), 226–229.
- [42] Seyed Saber Nabavi Larimi, Behzad Salami, Osman S Unsal, Adrián Cristal Kestelman, Hamid Sarbazi-Azad, and Onur Mutlu. 2021. Understanding power consumption and reliability of high-bandwidth memory with voltage underscaling. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 517–522.
- [43] Karl Leboeuf, Ashkan Hosseinzadeh Namin, Roberto Muscedere, Huapeng Wu, and Majid Ahmadi. 2008. High speed VLSI implementation of the hyperbolic tangent sigmoid function. In *2008 Third international conference on convergence and hybrid information technology*, Vol. 1. IEEE, 1070–1073.
- [44] John Leonard, Jonathan How, Seth Teller, Mitch Berger, Stefan Campbell, Gaston Fiore, Luke Fletcher, Emilio Frazzoli, Albert Huang, Sertac Karaman, et al. 2008. A perception-driven autonomous urban vehicle. *Journal of Field Robotics* 25, 10 (2008), 727–774.
- [45] Yuan Li, Ahmed Louri, and Avinash Karanth. 2022. SPACX: Silicon photonics-based scalable chiplet accelerator for DNN inference. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 831–845.
- [46] Xing Lin, Yair Rivenson, Nezhir T. Yardimci, Muhammed Veli, Yi Luo, Mona Jarrahi, and Aydogan Ozcan. 2018. All-optical machine learning using diffractive deep neural networks. *Science* 361 (9 2018), 1004–1008. Issue 6406. <https://doi.org/10.1126/science.aat8084>
- [47] Xiaoxiao Liu, Mengjie Mao, Beiyue Liu, Hai Li, Yiran Chen, Boxun Li, Yu Wang, Hao Jiang, Mark Barnell, Qing Wu, et al. 2015. RENO: A high-efficient reconfigurable neuromorphic computing accelerator design. In *Proceedings of the 52nd Annual Design Automation Conference*. 1–6.
- [48] RT McAllister, Yarin Gal, Alex Kendall, Mark Van Der Wilk, Amar Shah, Roberto Cipolla, and Adrian Weller. 2017. Concrete problems for autonomous vehicle safety: Advantages of Bayesian deep learning. *International Joint Conferences on Artificial Intelligence*, Inc.
- [49] Pramod Kumar Meher. 2010. An optimized lookup-table for the evaluation of sigmoid function for artificial neural networks. In *2010 18th IEEE/IFIP International Conference on VLSI and System-on-Chip*. IEEE, 91–95.
- [50] Sakorn Mekruksavanich and Anuchit Jitpattanakul. 2021. Lstm networks using smartphone data for sensor-based human activity recognition in smart homes. *Sensors* 21, 5 (2021), 1636.
- [51] Sajjad Moazeni, Sen Lin, Mark Wade, Luca Alloatti, Rajeev J. Ram, Milos Popovic, and Vladimir Stojanovic. 2017. A 40-Gb/s PAM-4 Transmitter Based on a Ring-Resonator Optical DAC in 45-nm SOI CMOS. *IEEE Journal of Solid-State Circuits* 52 (12 2017), 3503–3516. Issue 12. <https://doi.org/10.1109/JSSC.2017.2748620>
- [52] Sina Najmaei, Andreu L Glasmann, Marshall A Schroeder, Wendy L Sarney, Matthew L Chin, and Daniel M Potrepka. 2022. Advancements in materials, devices, and integration schemes for a new generation of neuromorphic computers. *Materials Today* 59 (2022), 80–106.
- [53] Pritish Narayanan, Stefano Ambrogio, Atsuya Okazaki, Kohji Hosokawa, Hsin-yu Tsai, Akiyo Nomura, Takeo Yasuda, Charles Mackin, Scott C Lewis, Alexander Friz, et al. 2021. Fully on-chip MAC at 14 nm enabled by accurate row-wise programming of PCM-based weights and parallel vector-transport in duration-format. *IEEE Transactions on Electron Devices* 68, 12 (2021), 6629–6636.
- [54] James William Nilsson. 2000. *Introduction to PSpice® Manual, Electric Circuits: Using ORCad® Release 9.1*. Prentice Hall, Upper Saddle River, NJ.
- [55] Kengo Nozaki, Shinji Matsuo, Takuro Fujii, Koji Takeda, Masaaki Ono, Abdul Shakoor, Eiichi Kuramochi, and Masaya Notomi. 2016. Photonic-crystal nanophotodetector with ultrasmall capacitance for on-chip light-to-voltage conversion without an amplifier. *Optica* 3, 5 (May 2016), 483–492. <https://doi.org/10.1364/OPTICA.3.000483>
- [56] Cedric Nugteren. 2018. CLBlast: A tuned OpenCL BLAS library. In *Proceedings of the International Workshop on OpenCL*. 1–10.
- [57] Dong-Ryeol Oh, Kyoung-Jun Moon, Won-Mook Lim, Ye-Dam Kim, Eun-Ji An, and Seung-Tak Ryu. 2020. An 8-bit 1-GS/s asynchronous loop-unrolled SAR-flash ADC with complementary dynamic amplifiers in 28-nm CMOS. *IEEE Journal of Solid-State Circuits* 56, 4 (2020), 1216–1226.
- [58] BM Oliver. 1961. Signal to noise ratios in photoelectric mixing. *Proc. IRE* 49, 12 (1961), 1960.
- [59] Zhe Pan, Zonghua Gu, Xiaohong Jiang, Guoquan Zhu, and De Ma. 2022. A modular approximation methodology for efficient fixed-point hardware implementation of the sigmoid function. *IEEE Transactions on Industrial Electronics* 69, 10 (2022), 10694–10703.
- [60] Trishan Panch, Peter Szolovits, and Rifat Atun. 2018. Artificial intelligence, machine learning and health systems. *Journal of global health* 8, 2 (2018).
- [61] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [62] David Patel, Venkat Veerasubramanian, Samir Ghosh, Alireza Samani, Qihang Zhong, and David V. Plant. 2014. High-speed compact silicon photonic Michelson interferometric modulator. *Optics Express* 22 (11 2014), 26788. Issue 22. <https://doi.org/10.1364/OE.22.026788>
- [63] Marc'Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. 2015. Sequence level training with recurrent neural networks. *arXiv preprint arXiv:1511.06732* (2015).
- [64] Christopher Rogers, Alexander Y. Piggott, David J. Thomson, Robert F. Wiser, Ion E. Opris, Steven A. Fortune, Andrew J. Compston, Alexander Gondarenko, Fanfan Meng, Xia Chen, Graham T. Reed, and Remus Nicolaescu. 2021. A universal 3D imaging sensor on a silicon photonics platform. *Nature* 590 (2 2021), 256–261. Issue 7845. <https://doi.org/10.1038/s41586-021-03259-y>
- [65] R Saligram, A Raychowdhury, and Suman Datta. 2024. The future is frozen: cryogenic CMOS for high-performance computing. *Chip* 3, 1 (2024), 100082.
- [66] Emre Sezgin. 2023. Artificial intelligence in healthcare: Complementing, not replacing, doctors and healthcare providers. *Digital health* 9 (2023), 20552076231186520.
- [67] Vivswan Shah and Nathan Youngblood. 2023. AnalogVNN: A fully modular framework for modeling and optimizing photonic neural networks. *APL Machine Learning* 1 (6 2023). Issue 2. <https://doi.org/10.1063/5.0134156>
- [68] Yichen Shen, Nicholas C. Harris, Scott Skirlo, Mihika Prabhu, Tom Baehr-Jones, Michael Hochberg, Xin Sun, Shijie Zhao, Hugo Larochelle, Dirk Englund, and Marin Soljačić. 2017. Deep learning with coherent nanophotonic circuits. *Nature Photonics* 11 (6 2017), 441–446. Issue 7. <https://doi.org/10.1038/nphoton.2017.93>
- [69] Kyle Shiflett, Avinash Karanth, Razvan Bunescu, and Ahmed Louri. 2023. Flumen: Dynamic processing in the photonic interconnect. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
- [70] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
- [71] Alexander Sluuds, Saumil Bandyopadhyay, Zaijun Chen, Zhizhen Zhong, Jared Cochrane, Liane Bernstein, Darius Bunandar, P Ben Dixon, Scott A Hamilton, Matthew Shreshinsky, et al. 2022. Delocalized photonic deep learning on the internet's edge. *Science* 378, 6617 (2022), 270–276.
- [72] Shihao Song, Adarsha Balaji, Anup Das, and Nagarajan Kandasamy. 2022. Design-technology co-optimization for NVM-based neuromorphic processing elements. *ACM Transactions on Embedded Computing Systems* 21, 6 (2022), 1–27.
- [73] Nikita Stroeve and Natalia G Berloff. 2023. Analog photonics computing for information processing, inference, and optimization. *Advanced Quantum Technologies* 6, 9 (2023), 2300055.
- [74] Jie Sun, Erman Timurdogan, Ami Yaacobi, Ehsan Shah Hosseini, and Michael R. Watts. 2013. Large-scale nanophotonic phased array. *Nature* 493 (1 2013), 195–199. Issue 7431. <https://doi.org/10.1038/nature11727>
- [75] Micron Technology. [n. d.]. HBM2e DRAM. <https://www.micron.com/products/dram/hbm/hbm2e>. Accessed: 2025-02-16.
- [76] Neil C Thompson, Kristjan Greenewald, Keeheon Lee, and Gabriel F Manso. 2020. The computational limits of deep learning. *arXiv preprint arXiv:2007.05558* (2020).
- [77] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [78] Ivan Tsmots, Oleksa Skorokhoda, and Vasyil Rabyk. 2019. Hardware implementation of sigmoid activation functions using FPGA. In *2019 IEEE 15th International Conference on the Experience of Designing and Application of CAD Systems (CADSM)*. IEEE, 34–38.
- [79] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [80] Weier Wan, Rajkumar Kubendran, Clemens Schaefer, Sukru Burc Eryilmaz, Wenqiang Zhang, Dabin Wu, Stephen Deiss, Priyanka Raina, He Qian, Bin Gao, et al. 2022. A compute-in-memory chip based on resistive random-access memory. *Nature* 608, 7923 (2022), 504–512.
- [81] Changming Wu, Haoqin Deng, Yi-Siou Huang, Heshan Yu, Ichiro Takeuchi, Carlos A Rios Ocampo, and Mo Li. 2024. Freeform direct-write and rewritable photonic integrated circuits in phase-change thin films. *Science Advances* 10, 1 (2024), eadk1361.
- [82] Hao Wu, Patrick Judd, Xiaojie Zhang, Mikhail Isaev, and Paulius Micikevicius. 2020. Integer quantization for deep learning inference: Principles and empirical evaluation. *arXiv preprint arXiv:2004.09602* (2020).
- [83] Xiaotie Wu, Bipin Dama, Prakash Gothoskar, Peter Metz, Kal Shastri, Sanjay Sunder, Jan Van der Spiegel, Yifan Wang, Mark Webster, and Will Wilson. 2013. A 20Gb/s NRZ/PAM-4 1V transmitter in 40nm CMOS driving a Si-photonic modulator in 0.13μm CMOS. *2013 IEEE International Solid-State Circuits Conference Digest of Technical Papers*, 128–129. <https://doi.org/10.1109/ISSCC.2013.6487667>
- [84] Xiaotie Wu, Bipin Dama, Prakash Gothoskar, Peter Metz, Kal Shastri, Sanjay Sunder, Jan Van der Spiegel, Yifan Wang, Mark Webster, and Will Wilson. 2013. A 20Gb/s NRZ/PAM-4 1V transmitter in 40nm CMOS driving a Si-photonic modulator in 0.13 μm CMOS. In *2013 IEEE International Solid-State Circuits Conference Digest of Technical Papers*. IEEE, 128–129.
- [85] Xin Xin, Yanan Guo, Youtao Zhang, and Jun Yang. 2021. SAM: Accelerating Strided Memory Accesses. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (Virtual Event, Greece) (MICRO '21)*. Association

- for Computing Machinery, New York, NY, USA, 324–336. <https://doi.org/10.1145/3466752.3480091>
- [86] Guowei Yang, Cansu Demirkiran, Zeynep Ece Kizilates, Carlos A Ríos Ocampo, Ayse K Coskun, and Ajay Joshi. 2023. Processing-in-memory using optically-addressed phase change memory. In *2023 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 1–6.
- [87] Jong-Hyeok Yoon, Muya Chang, Win-San Khwa, Yu-Der Chih, Meng-Fan Chang, and Arijit Raychowdhury. 2021. 29.1 A 40nm 64Kb 56.67 TOPS/W read-disturb-tolerant compute-in-memory/digital RRAM macro with active-feedback-based read and in-situ write verification. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 64. IEEE, 404–406.
- [88] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. 2023. Zeus: Understanding and Optimizing {GPU} Energy Consumption of {DNN} Training. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 119–139.
- [89] Nathan Youngblood. 2023. Coherent Photonic Crossbar Arrays for Large-Scale Matrix-Matrix Multiplication. *IEEE Journal of Selected Topics in Quantum Electronics* 29, 2: Optical Computing (2023), 1–11. <https://doi.org/10.1109/JSTQE.2022.3171167>
- [90] Horace P Yuen and Vincent WS Chan. 1983. Noise in homodyne and heterodyne detection. *Optics letters* 8, 3 (1983), 177–179.
- [91] Babak Zamanlooy and Mitra Mirhassani. 2016. An analog CVNS-based sigmoid neuron for precise neurochips. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25, 3 (2016), 894–906.
- [92] H. Zhang, M. Gu, X. D. Jiang, J. Thompson, H. Cai, S. Paesani, R. Santagati, A. Laing, Y. Zhang, M. H. Yung, Y. Z. Shi, F. K. Muhammad, G. Q. Lo, X. S. Luo, B. Dong, D. L. Kwong, L. C. Kwek, and A. Q. Liu. 2021. An optical neural chip for implementing complex-valued neural network. *Nature Communications* 12 (12 2021), 457. Issue 1. <https://doi.org/10.1038/s41467-020-20719-7>
- [93] Xiaosheng Zhang, Kyungmok Kwon, Johannes Henriksson, Jianheng Luo, and Ming C. Wu. 2022. A large-scale microelectromechanical-systems-based silicon photonics LiDAR. *Nature* 603 (3 2022), 253–258. Issue 7900. <https://doi.org/10.1038/s41586-022-04415-8>